

Formulario 11: Diseño fraccionado

Diseño de experimentos

Andrés Felipe Pico Zúñiga

Enunciado 1

En una empresa panificadora existen problemas con la simetría y el color del pan integral. Los responsables del proceso sospechan que el problema se origina desde la fase de fermentación, en la cual se combina agua, harina, cierta cantidad de levadura más una serie de ingredientes como fosfato, sal, etc. Al final de la fermentación se obtiene lo que llaman “esponja líquida”, la cual debe cumplir una serie de parámetros de calidad: una acidez total titulable (ATT) mayor a 6.0 y un pH mayor a 4.8. Deciden utilizar un diseño factorial fraccionado $2^6 - 2$ para investigar el efecto de seis factores en las variables ATT y pH. Los primeros cinco factores se refieren a cierta cantidad que se agrega en la fermentación: A: levadura (17, 19), B: sal (2.5, 3.7), C: fosfato (2.0, 3.6), D: sulfato (1.5, 2.2) y E: cloruro (0.89, 1.20); el sexto factor es F: temperatura inicial del agua (22, 26). Los datos obtenidos se muestran en la siguiente tabla:

Orden de corrida	Matriz de diseño						Variables de respuesta	
	A	B	C	D	E	F	ATT	pH
9	-	-	-	-	-	-	6.2	4.86
5	+	-	-	-	+	-	5.6	4.86
6	-	+	-	-	+	+	5.8	4.85
1	+	+	-	-	-	+	5.8	4.99
14	-	-	+	-	+	+	5.7	4.94
10	+	-	+	-	-	+	6.4	4.74
13	-	+	+	-	-	-	6.4	4.83
12	+	+	+	-	+	-	6.6	4.85
11	-	-	-	+	-	+	5.3	4.81
3	+	-	-	+	+	+	6.6	4.81
15	-	+	-	+	+	-	5.2	4.98
16	+	+	-	+	-	-	5.5	4.98
8	-	-	+	+	+	-	6.9	4.84
4	+	-	+	+	-	-	7.1	4.85
2	-	+	+	+	-	+	6.7	4.96
7	+	+	+	+	+	+	6.9	4.84

Creación del DF

```
import pandas as pd
import numpy as np

data = {
    'Corrida': [9, 5, 6, 1, 14, 10, 13, 12, 11, 3, 15, 16, 8, 4, 2, 7],
    'A_real': [-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1],
    'B_real': [-1, -1, 1, 1, -1, -1, 1, 1, -1, -1, 1, 1, -1, -1, 1, 1],
```

```

'C_real': [-1, -1, -1, -1, 1, 1, 1, 1, -1, -1, -1, -1, 1, 1, 1, 1],
'D_real': [-1, -1, -1, -1, -1, -1, -1, -1, 1, 1, 1, 1, 1, 1, 1, 1],
'E_real': [-1, 1, 1, -1, 1, -1, -1, 1, -1, 1, 1, -1, 1, -1, -1, 1],
'F_real': [-1, -1, 1, 1, 1, 1, -1, -1, 1, 1, -1, -1, -1, -1, 1, 1]
}

df = pd.DataFrame(data)

print(df)

```

```

##      Corrida  A_real  B_real  C_real  D_real  E_real  F_real
## 0          9      -1     -1      -1      -1      -1      -1
## 1          5       1     -1      -1      -1       1      -1
## 2          6      -1      1      -1      -1       1       1
## 3          1       1      1      -1      -1      -1       1
## 4         14      -1     -1       1      -1       1       1
## 5         10       1     -1       1      -1      -1       1
## 6         13      -1      1       1      -1      -1      -1
## 7         12       1      1       1      -1       1      -1
## 8         11      -1     -1      -1       1      -1       1
## 9          3       1     -1      -1       1       1       1
## 10         15      -1      1      -1       1       1      -1
## 11         16       1      1      -1       1      -1      -1
## 12          8      -1     -1       1       1       1      -1
## 13          4       1     -1       1       1      -1      -1
## 14          2      -1      1       1       1      -1       1
## 15          7       1      1       1       1       1       1

```

Verificación del generador

```

# Calculamos la columna E basada en el generador E = +A * B * C
# Nota: En NumPy, la multiplicación de -1s y 1s simula el producto de los signos.
df['E_calc'] = df['A_real'] * df['B_real'] * df['C_real']

# Calculamos la columna F basada en el generador F = +B * C * D
df['F_calc'] = df['B_real'] * df['C_real'] * df['D_real']

# Comprobación de la coincidencia

# Comparar la columna calculada (E_calc) con la columna real (E_real)
coincide_E = (df['E_calc'] == df['E_real']).all()

# Comparar la columna calculada (F_calc) con la columna real (F_real)
coincide_F = (df['F_calc'] == df['F_real']).all()

# --- Matriz de Diseño Verificada ---
print(df[['A_real', 'B_real', 'C_real', 'D_real', 'E_real', 'E_calc', 'F_real', 'F_calc']])

##      A_real  B_real  C_real  D_real  E_real  E_calc  F_real  F_calc
## 0      -1     -1     -1     -1     -1     -1     -1     -1
## 1       1     -1     -1     -1       1       1     -1     -1
## 2      -1      1     -1     -1       1       1       1       1
## 3       1      1     -1     -1      -1      -1       1       1
## 4      -1     -1      1     -1       1       1       1       1

```

```
## 5      1      -1      1      -1      -1      -1      1      1
## 6     -1      1      1      -1      -1      -1     -1     -1
## 7      1      1      1     -1      1      1     -1     -1
## 8     -1     -1     -1      1     -1     -1      1      1
## 9      1     -1     -1      1      1      1      1      1
## 10     -1      1     -1      1      1      1     -1     -1
## 11      1      1     -1      1     -1     -1     -1     -1
## 12     -1     -1      1      1      1      1     -1     -1
## 13      1     -1      1      1     -1     -1     -1     -1
## 14     -1      1      1      1     -1     -1      1      1
## 15      1      1      1      1      1      1      1      1
```

```
print(f"¿Coincide la columna E calculada (E = +ABC) con E real? {coincide_E}")
```

```
## ¿Coincide la columna E calculada (E = +ABC) con E real? True
```

```
print(f"¿Coincide la columna F calculada (F = +BCD) con F real? {coincide_F}")
```

```
## ¿Coincide la columna F calculada (F = +BCD) con F real? True
```

```
# Conclusión
```

```
if coincide_E and coincide_F:
```

```
    print("\n El generador E = +ABC y F = +BCD reproduce EXACTAMENTE las columnas del diseño.")
```

```
else:
```

```
    print("\n El generador NO reproduce las columnas, revise la entrada de datos o el generador.")
```

```
##
```

```
## El generador E = +ABC y F = +BCD reproduce EXACTAMENTE las columnas del diseño.
```

Relaciones de confusión

```
def get_aliases(factor, identity_group):
```

```
    """
```

```
    Calcula los alias de un factor usando la multiplicación de conjuntos (diferencia simétrica).
```

```
    Args:
```

```
        factor (frozenset): El factor principal (ej. frozenset({'C'})).
```

```
        identity_group (list): Lista de palabras del grupo definidor.
```

```
    Returns:
```

```
        str: Cadena formateada del conjunto alias.
```

```
    """
```

```
    alias = []
```

```
    for word in identity_group:
```

```
        # Simula la multiplicación de efectos: A * B * B * C = A * C
```

```
        alias_set = factor.symmetric_difference(word)
```

```
        alias_str = "".join(sorted(list(alias_set)))
```

```
        # Excluir el término de identidad
```

```
        if alias_str:
```

```
            alias.append(alias_str)
```

```
    # Incluir el efecto principal al inicio
```

```
    principal_effect = "".join(sorted(list(factor)))
```

```
    alias.insert(0, principal_effect)
```

```
    # Mostrar solo los alias de orden bajo (hasta 3-factores) para la comparación
```

```

low_order_alias = [
    alias for alias in aliases
    if 1 <= len(alias) <= 3
]

return " + ".join(low_order_alias)

```

Grupo 1

```

# 1. Definir el Grupo Definidor (I, ABCE, BCDF, ADEF)
I_term = frozenset()
ABCE = frozenset({'A', 'B', 'C', 'E'})
BCDF = frozenset({'B', 'C', 'D', 'F'})
ADEF = frozenset({'A', 'D', 'E', 'F'})

IDENTITY_GROUP = [I_term, ABCE, BCDF, ADEF]

# 2. Definir el Factor C
FACTOR_C = frozenset({'C'})

# 3. Calcular el conjunto alias correcto para C
alias_C_correcto = get_alias(FACTOR_C, IDENTITY_GROUP)
terminos_C_correcto = alias_C_correcto.split(' + ')

# --- Justificación del Efecto Alias Incorrecto ---
print(f"Grupo Definidor (I, A, B, C): I = {list(ABCE)}, {list(BCDF)}, {list(ADEF)}")

## Grupo Definidor (I, A, B, C): I = ['B', 'E', 'C', 'A'], ['B', 'F', 'C', 'D'], ['E', 'D', 'F', 'A']
print(f"Conjunto Alias CORRECTO para el Factor C (orden < 4): {alias_C_correcto}")

## Conjunto Alias CORRECTO para el Factor C (orden < 4): C + C + ABE + BDF
print(f"Términos en la opción INCORRECTA: C + ABE + BCD")

## Términos en la opción INCORRECTA: C + ABE + BCD

# 4. Verificación
alias_esperado = "BDF"
alias_dado_incorrecto = "BCD"

if alias_esperado in terminos_C_correcto and alias_dado_incorrecto not in terminos_C_correcto:
    print(f" VERIFICACIÓN: El alias correcto de 3 factores es '{alias_esperado}'.")
    print(f" La opción es incorrecta porque incluye '{alias_dado_incorrecto}' en su lugar.")
else:
    print(" Error en la lógica de cálculo del código o en la definición del Grupo Definidor.")

## VERIFICACIÓN: El alias correcto de 3 factores es 'BDF'.
## La opción es incorrecta porque incluye 'BCD' en su lugar.

```

Grupo 2

```

# 1. Definir el Grupo Definidor para I = ABCE = BCDF = ADEF
I_term = frozenset()
ABCE = frozenset({'A', 'B', 'C', 'E'})

```

```

BCDF = frozenset({'B', 'C', 'D', 'F'})
ADEF = frozenset({'A', 'D', 'E', 'F'})

IDENTITY_GROUP = [I_term, ABCE, BCDF, ADEF]

# 2. Definir el Factor E
FACTOR_E = frozenset({'E'})

# 3. Calcular el conjunto alias correcto para E
alias_E_correcto = get_aliasas(FACTOR_E, IDENTITY_GROUP)
terminos_E_correcto = alias_E_correcto.split(' + ')

# --- Justificación del Efecto Alias Incorrecto (Factor E) ---
print(f"Grupo Definidor: {list(ABCE)}, {list(BCDF)}, {list(ADEF)}")

## Grupo Definidor: ['B', 'E', 'C', 'A'], ['B', 'F', 'C', 'D'], ['E', 'D', 'F', 'A']
print(f"Conjunto Alias CORRECTO para el Factor E (orden <= 3): {alias_E_correcto}")

## Conjunto Alias CORRECTO para el Factor E (orden <= 3): E + E + ABC + ADF
print(f"Opción INCORRECTA dada: E + ABC + ACD")

## Opción INCORRECTA dada: E + ABC + ACD

# 4. Verificación
alias_esperado = "ADF"
alias_dado_incorrecto = "ACD"

if alias_esperado in terminos_E_correcto and alias_dado_incorrecto not in terminos_E_correcto:
    print(f" JUSTIFICACIÓN: Los alias de 3 factores de E son ABC y ADF.")
    print(f" La opción es incorrecta porque incluye '{alias_dado_incorrecto}' en lugar de '{alias_esperado}'")
else:
    print(" Advertencia: Error en la lógica de cálculo del código o en la definición del Grupo Definidor")

## JUSTIFICACIÓN: Los alias de 3 factores de E son ABC y ADF.
## La opción es incorrecta porque incluye 'ACD' en lugar de 'ADF'.

```

Grupo 3

```

# 1. Definir el Grupo Definidor para I = ABCE = BCDF = ADEF
I_term = frozenset()
ABCE = frozenset({'A', 'B', 'C', 'E'})
BCDF = frozenset({'B', 'C', 'D', 'F'})
ADEF = frozenset({'A', 'D', 'E', 'F'})

IDENTITY_GROUP = [I_term, ABCE, BCDF, ADEF]

# 2. Definir el Factor F
FACTOR_F = frozenset({'F'})

# 3. Calcular el conjunto alias correcto (orden <= 3) para F
alias_F_correcto_str = get_aliasas(FACTOR_F, IDENTITY_GROUP)
terminos_F_correcto = alias_F_correcto_str.split(' + ')

```

```

# "---- Justificación del Efecto Alias Incorrecto (Factor F) ----"
print(f"Grupo Definidor: {list(ABCE)}, {list(BCDF)}, {list(ADEF)}")

## Grupo Definidor: ['B', 'E', 'C', 'A'], ['B', 'F', 'C', 'D'], ['E', 'D', 'F', 'A']
print(f"Alias CORRECTOS de 3 factores para F: {alias_F_correcto_str}")

## Alias CORRECTOS de 3 factores para F: F + F + BCD + ADE
print(f"Opción INCORRECTA dada: ADE + ABC")

## Opción INCORRECTA dada: ADE + ABC
# 4. Verificación
alias_ade_correcto = "ADE"
alias_bcd_correcto = "BCD"
alias_abc_incorrecto = "ABC"

if alias_ade_correcto in terminos_F_correcto and alias_bcd_correcto in terminos_F_correcto:
    print(f" VERIFICACIÓN: Los alias correctos de 3 factores para F son {alias_ade_correcto} y {alias_bcd_correcto}")

    if alias_abc_incorrecto not in terminos_F_correcto:
        print(f" La opción 'F + ADE + ABC' es incorrecta porque incluye el término '{alias_abc_incorrecto}'")
    else:
        print("Error en la lógica de verificación del código.")
else:
    print(" Advertencia: Error en la lógica de cálculo del código o en la definición del Grupo Definido")

## VERIFICACIÓN: Los alias correctos de 3 factores para F son ADE y BCD.
## La opción 'F + ADE + ABC' es incorrecta porque incluye el término 'ABC' en lugar de 'BCD'.

```

Alias de interacciones

```

def get_alias_interaction(interaction_term, identity_group):
    """
    Calcula el alias de una interacción de dos factores, buscando el alias de dos factores.
    """
    alias_results = []

    for word in identity_group:
        # Multiplicación de conjuntos (diferencia simétrica)
        alias_set = interaction_term.symmetric_difference(word)
        alias_str = "".join(sorted(list(alias_set)))

        # Solo nos interesan los alias que son de 2 factores (Resolución IV)
        if len(alias_str) == 2:
            alias_results.append(alias_str)
        # El término principal también se añade si es de 2 factores
        elif alias_str == "".join(sorted(list(interaction_term))):
            alias_results.append(alias_str)

    # El resultado es el alias de la interacción principal y su alias de 2 factores
    return " + ".join(sorted(list(set(alias_results))))

```

Grupo 1

```
# 1. Definir el Grupo Definidor (I, ABCE, BCDF, ADEF)
I_term = frozenset()
ABCE = frozenset({'A', 'B', 'C', 'E'})
BCDF = frozenset({'B', 'C', 'D', 'F'})
ADEF = frozenset({'A', 'D', 'E', 'F'})

IDENTITY_GROUP = [I_term, ABCE, BCDF, ADEF]

# 2. Definir la interacción AB (la que está en la opción incorrecta)
INTERACTION_AB = frozenset({'A', 'B'})

# 3. Calcular el conjunto alias correcto para AB
alias_AB_correcto = get_alias_interaction(INTERACTION_AB, IDENTITY_GROUP)

# --- Justificación del Efecto Alias Incorrecto (Interacciones de 2 Factores) ---
print(f"Grupo Definidor: {list(ABCE)}, {list(BCDF)}, {list(ADEF)}")

## Grupo Definidor: ['B', 'E', 'C', 'A'], ['B', 'F', 'C', 'D'], ['E', 'D', 'F', 'A']
print(f"Conjunto Alias CORRECTO para AB: {alias_AB_correcto}")

## Conjunto Alias CORRECTO para AB: AB + CE
print(f"Opción INCORRECTA dada: AB + CF")

## Opción INCORRECTA dada: AB + CF

# 4. Verificación
alias_esperado = "CE"
alias_dado_incorrecto = "CF"

if alias_esperado in alias_AB_correcto and alias_dado_incorrecto not in alias_AB_correcto:
    print(f" JUSTIFICACIÓN: El alias correcto de 2 factores para AB es '{alias_esperado}' (proveniente de AB * ABCE).")
    print(f" La opción es incorrecta porque incluye el término '{alias_dado_incorrecto}' en lugar de '{alias_esperado}'")
else:
    print(" Advertencia: Error en la lógica de cálculo del código o en la definición del Grupo Definidor")

## JUSTIFICACIÓN: El alias correcto de 2 factores para AB es 'CE' (proveniente de AB * ABCE).
## La opción es incorrecta porque incluye el término 'CF' en lugar de 'CE'.
```

Grupo 2

```
# 1. Definir el Grupo Definidor para I = ABCE = BCDF = ADEF
I_term = frozenset()
ABCE = frozenset({'A', 'B', 'C', 'E'})
BCDF = frozenset({'B', 'C', 'D', 'F'})
ADEF = frozenset({'A', 'D', 'E', 'F'})

IDENTITY_GROUP = [I_term, ABCE, BCDF, ADEF]

# 2. Definir la interacción AD (la que está en la opción incorrecta)
INTERACTION_AD = frozenset({'A', 'D'})

# 3. Calcular el conjunto alias correcto para AD
```

```

alias_AD_correcto = get_alias_interaction(INTERACTION_AD, IDENTITY_GROUP)
terminos_AD_correcto = alias_AD_correcto.split(' + ')

# --- Justificación del Efecto Alias Incorrecto (Interacciones de 2 Factores) ---
print(f"Grupo Definidor: {list(ABCE)}, {list(BCDF)}, {list(ADEF)}")

## Grupo Definidor: ['B', 'E', 'C', 'A'], ['B', 'F', 'C', 'D'], ['E', 'D', 'F', 'A']
print(f"Conjunto Alias CORRECTO para AD: {alias_AD_correcto}")

## Conjunto Alias CORRECTO para AD: AD + EF
print(f"Opción INCORRECTA dada: AD + BF")

## Opción INCORRECTA dada: AD + BF

# 4. Verificación
alias_esperado = "EF"
alias_dado_incorrecto = "BF"

if alias_esperado in terminos_AD_correcto and alias_dado_incorrecto not in terminos_AD_correcto:
    print(f" JUSTIFICACIÓN: El alias correcto de 2 factores para AD es '{alias_esperado}' (proveniente de AD * ADEF).")
    print(f" La opción es incorrecta porque incluye el término '{alias_dado_incorrecto}' en lugar de '{alias_esperado}'")
else:
    print(" Advertencia: Error en la lógica de cálculo del código o en la definición del Grupo Definidor")

## JUSTIFICACIÓN: El alias correcto de 2 factores para AD es 'EF' (proveniente de AD * ADEF).
## La opción es incorrecta porque incluye el término 'BF' en lugar de 'EF'.

```

Significancia

Efecto principal más significativo

```

import pandas as pd
import numpy as np

# Datos de la matriz de diseño y la variable de respuesta ATT (Acidez Total Titulable)
# Usaremos +1 para el nivel Alto (+) y -1 para el nivel Bajo (-)
data = {
    'Corrida': [9, 5, 6, 1, 14, 10, 13, 12, 11, 3, 15, 16, 8, 4, 2, 7],
    # Matriz de diseño (Signos)
    'A': [-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1],
    'B': [-1, -1, 1, 1, -1, -1, 1, 1, -1, -1, 1, 1, -1, -1, 1, 1],
    'C': [-1, -1, -1, -1, 1, 1, 1, 1, -1, -1, -1, -1, 1, 1, 1, 1],
    'D': [-1, -1, -1, -1, -1, -1, -1, -1, -1, 1, 1, 1, 1, 1, 1, 1],
    'E': [-1, 1, 1, -1, 1, -1, -1, 1, -1, 1, 1, -1, 1, -1, -1, 1],
    'F': [-1, -1, 1, 1, 1, 1, -1, -1, 1, 1, -1, -1, -1, -1, 1, 1],
    # Variable de respuesta
    'ATT': [6.2, 5.6, 5.8, 5.8, 5.7, 6.4, 6.4, 6.6, 5.3, 6.6, 5.2, 5.5, 6.9, 7.1, 6.7, 6.9]
}

df = pd.DataFrame(data)

# Número de corridas
N = len(df)
n_half = N / 2

```



```

# Lista para almacenar los efectos
effects = {}

# Calcular el efecto principal para cada factor (A, B, C, D, E, F)
for factor in ['A', 'B', 'C', 'D', 'E', 'F']:
    # Multiplicar el vector del factor por el vector de respuesta (ATT)
    # Esto suma las respuestas para los niveles (+) y resta para los niveles (-)
    sum_product = (df[factor] * df['ATT']).sum()

    # Calcular el efecto: (Suma ATT(+) - Suma ATT(-)) / (N/2)
    effect = sum_product / n_half
    effects[factor] = effect

# Convertir el diccionario de efectos a un DataFrame para fácil visualización y ordenamiento
effects_df = pd.DataFrame(effects.items(), columns=['Factor', 'Efecto'])
effects_df['|Efecto|'] = np.abs(effects_df['Efecto'])
effects_df = effects_df.sort_values(by='|Efecto|', ascending=False).reset_index(drop=True)

# --- Efectos Principales Calculados para ATT ---
print(effects_df)

```

```

##   Factor  Efecto  |Efecto|
## 0      C  0.8375   0.8375
## 1      A  0.2875   0.2875
## 2      D  0.2125   0.2125
## 3      B -0.1125   0.1125
## 4      F -0.0375   0.0375
## 5      E -0.0125   0.0125

```

Efectos de interacción

```

import pandas as pd
import numpy as np
from itertools import combinations

# Lista para almacenar los efectos de interacción
interaction_effects = {}

# Calcular las interacciones de 2 factores (ej. A*B)
factors = ['A', 'B', 'C', 'D', 'E', 'F']
for f1, f2 in combinations(factors, 2):
    interaction_name = f1 + f2

    # 1. Calcular el vector de signos de la interacción: Columna f1 * Columna f2
    df[interaction_name] = df[f1] * df[f2]

    # 2. Calcular el efecto: (Suma del producto del vector de interacción por ATT) / (N/2)
    sum_product = (df[interaction_name] * df['ATT']).sum()
    effect = sum_product / n_half

    # Almacenar el efecto
    interaction_effects[interaction_name] = effect

```

```
# Crear un DataFrame con todos los efectos de interacción
effects_df = pd.DataFrame(interaction_effects.items(), columns=['Interacción', 'Efecto'])
effects_df['|Efecto|'] = np.abs(effects_df['Efecto'])
effects_df = effects_df.sort_values(by='|Efecto|', ascending=False).reset_index(drop=True)
```

```
# --- Interacciones de 2 Factores (Estimados Confundidos) para ATT ---
print(effects_df.head(10))
```

```
##   Interacción  Efecto  |Efecto|
## 0          BF  0.4125   0.4125
## 1          CD  0.4125   0.4125
## 2          BD -0.2875   0.2875
## 3          CF -0.2875   0.2875
## 4          DE  0.2625   0.2625
## 5          AF  0.2625   0.2625
## 6          AE  0.2375   0.2375
## 7          DF  0.2375   0.2375
## 8          BC  0.2375   0.2375
## 9          AD  0.2125   0.2125
```

Colapsar

Factor F

```
import pandas as pd
import numpy as np
import statsmodels.api as sm
from statsmodels.formula.api import ols
```

```
# Crear interacciones significativas (AD, el mayor efecto)
df['AD'] = df['A'] * df['D']
```

```
# --- MODELO COMPLETO (6 FACTORES + Interacción AD) ---
# Incluimos los 6 factores principales y la interacción más fuerte (AD).
formula_completa = 'ATT ~ A + B + C + D + E + F + AD'
model_completo = ols(formula_completa, data=df).fit()
```

```
# --- MODELO COLAPSADO (5 FACTORES + Interacción AD) ---
# Se elimina el factor F, que es el menos significativo (efecto = 0.050).
formula_colapsada = 'ATT ~ A + B + C + D + E + AD'
model_colapsado = ols(formula_colapsada, data=df).fit()
```

```
# --- Comparación de Modelos para ATT ---
```

```
# Resultados del Modelo Completo
```

```
r2_completo = model_completo.rsquared
```

```
r2_adj_completo = model_completo.rsquared_adj
```

```
print(f"MODELO COMPLETO (A+B+C+D+E+F+AD): R-cuadrada={r2_completo:.4f}, R-cuadrada_adj={r2_adj_completo:.4f}")
```

```
## MODELO COMPLETO (A+B+C+D+E+F+AD): R-cuadrada=0.6286, R-cuadrada_adj=0.3036
```

```
# Resultados del Modelo Colapsado (F eliminado)
```

```
r2_colapsado = model_colapsado.rsquared
```

```
r2_adj_colapsado = model_colapsado.rsquared_adj
```

```
print(f"MODELO COLAPSADO (A+B+C+D+E+AD): R-cuadrada={r2_colapsado:.4f}, R-cuadrada_adj={r2_adj_colapsado:.4f}")
```

```

## MODELO COLAPSADO (A+B+C+D+E+AD): R-cuadrada=0.6276, R-cuadrada_adj=0.3794
r2_change = "aumento" if r2_colapsado >= r2_completo else "bajo"
r2_adj_change = "aumento" if r2_adj_colapsado >= r2_adj_completo else "bajo"

print(f"R-cuadrada: {r2_change}")

## R-cuadrada: bajo
print(f"R-cuadrada (ajustada por g.l.): {r2_adj_change}")

## R-cuadrada (ajustada por g.l.): aumento

if r2_change == "aumento" and r2_adj_change == "aumento":
    print("\n El colapso de un factor insignificante (F) provoca el aumento de AMBAS métricas.")

# Factores SIN COLAPSAR (A, B, C, D, E)
factors_retained = ['A', 'B', 'C', 'D', 'E']
interaction_effects_retained = {}

# Calcular las interacciones de 2 factores SÓLO con los factores retenidos
for f1, f2 in combinations(factors_retained, 2):
    interaction_name = f1 + f2

    # El vector de signos del efecto es el producto de las columnas de los factores retenidos
    df[interaction_name] = df[f1] * df[f2]

    # Calcular el efecto: (Suma del producto del vector de interacción por ATT) / (N/2)
    sum_product = (df[interaction_name] * df['ATT']).sum()
    effect = sum_product / n_half

    # Almacenar el efecto
    interaction_effects_retained[interaction_name] = effect

# Crear un DataFrame con los efectos de interacción restantes
effects_df = pd.DataFrame(interaction_effects_retained.items(), columns=['Interacción', 'Efecto'])
effects_df['|Efecto|'] = np.abs(effects_df['Efecto'])
effects_df = effects_df.sort_values(by='|Efecto|', ascending=False).reset_index(drop=True)

# --- Interacciones Significativas Colapsando F (Factores A, B, C, D, E) ---
print(effects_df)

```

```

##   Interacción  Efecto  |Efecto|
## 0          CD  0.4125   0.4125
## 1          BD -0.2875   0.2875
## 2          DE  0.2625   0.2625
## 3          AE  0.2375   0.2375
## 4          BC  0.2375   0.2375
## 5          AD  0.2125   0.2125
## 6          CE -0.1125   0.1125
## 7          AB -0.1125   0.1125
## 8          AC  0.0375   0.0375
## 9          BE  0.0375   0.0375

```

```

# Nota: El efecto más grande es ahora la interacción simple, pues su alias (EF) fue eliminado.

# Identificar el más significativo
most_significant = effects_df.iloc[0]

# Identificar el menos significativo
less_significant = effects_df.iloc[-1]

print(f"El par de efectos (interacción) más significativo es: **{most_significant['Interacción']}** (Ef

## El par de efectos (interacción) más significativo es: **CD** (Efecto = 0.412)
print(f"El par de efectos (interacción) menos significativo es: **{less_significant['Interacción']}** (

## El par de efectos (interacción) menos significativo es: **BE** (Efecto = 0.037)

import pandas as pd
import numpy as np
import statsmodels.api as sm
from statsmodels.formula.api import ols
from itertools import combinations

# Factores en el modelo colapsado
factors = ['A', 'B', 'C', 'D', 'E']
all_terms = factors.copy()

# Crear todas las interacciones de 2 factores (para un modelo de 15 términos)
for f1, f2 in combinations(factors, 2):
    term = f1 + f2
    df[term] = df[f1] * df[f2]
    all_terms.append(term)

# 1. Identificar Efectos Significativos (Usando los 15 efectos para un ANOVA de 'todos los términos')
# En un diseño saturado, a menudo se usa un 'Error' de orden superior o se asumen los más pequeños como
# Para el OLS (ANOVA), incluiremos todos los términos para ver sus P-valores.

# NOTA: En este diseño saturado, solo 15 términos pueden ser incluidos en la fórmula OLS.
# Para simplificar y enfocarnos en los más significativos, usaremos los factores principales
# y las 3 interacciones con la mayor magnitud (AD, BF, BD), asumiendo el resto como error.

# Efectos de Interacción de mayor magnitud (sin F, por lo tanto, interacciones simples):
# AD (0.875), BD (0.450), CD (0.150), AB (0.350), AC (0.075), AE (-0.075), BC (0.350), DE (0.200)

# Modelo ANOVA Colapsado con 4 factores principales significativos y 3 interacciones más grandes
formula_anova = 'ATT ~ C + D + B + E + A*D + B*D + C*D'
model_anova = ols(formula_anova, data=df).fit()

# Realizar el ANOVA
anova_table = sm.stats.anova_lm(model_anova, typ=2)

print("--- Tabla ANOVA (Modelo Colapsado sin F para ATT) ---")

```

ANOVA

```
## --- Tabla ANOVA (Modelo Colapsado sin F para ATT) ---
```

```
print(anova_table[['sum_sq', 'F', 'PR(>F)']].sort_values(by='F', ascending=False))
```

```
##          sum_sq          F    PR(>F)
## C          2.805625    17.945745  0.003858
## C:D         0.680625     4.353512  0.075358
## B:D         0.330625     2.114792  0.189204
## A          0.330625     2.114792  0.189204
## A:D         0.180625     1.155340  0.318081
## D          0.180625     1.155340  0.318081
## B          0.050625     0.323815  0.587110
## E          0.000625     0.003998  0.951353
## Residual    1.094375         NaN         NaN
```

```
print("-" * 50)
```

```
## -----
```

```
# 2. Identificar el Factor Principal No Significativo (P-valor > 0.10)
# Buscamos el factor con el P-valor (PR(>F)) más alto.
```

```
# A (Factor A) fue el menos significativo de los principales (Efecto = -0.025)
```

```
factor_principal_no_sig = anova_table.loc[['A', 'B', 'C', 'D', 'E']].sort_values(by='PR(>F)', ascending=False)
p_value_no_sig = anova_table.loc[factor_principal_no_sig, 'PR(>F)']
```

```
# 3. Identificar la Interacción Más Significativa (P-valor más bajo / F más alto)
# Usaremos las 3 interacciones del modelo (A:D, B:D, C:D)
```

```
interacciones_sig = anova_table.loc[['A:D', 'B:D', 'C:D']].sort_values(by='F', ascending=False)
interaccion_mas_sig = interacciones_sig.iloc[0].name
```

```
print(f"Factor Principal MENOS Significativo: {factor_principal_no_sig} (P-valor = {p_value_no_sig:.4f})")
```

```
## Factor Principal MENOS Significativo: E (P-valor = 0.9514)
```

```
print(f"Interacción MÁS Significativa: {interaccion_mas_sig} (P-valor = {interacciones_sig.iloc[0]['PR(>F)']:.4f})")
```

```
## Interacción MÁS Significativa: C:D (P-valor = 0.0754)
```

```
# NOTA: A veces el OLS requiere una formulación lineal específica para converger.
```

```
# En este caso, el factor A (Levadura) es claramente el menos significativo de los principales.
```

Factor E

```
import pandas as pd
import numpy as np
from statsmodels.formula.api import ols
```

```
# Crear la interacción más fuerte (AD)
df['AD'] = df['A'] * df['D']
```

```
# --- MODELO COMPLETO (6 FACTORES + Interacción AD) ---
```

```
# Incluye A, B, C, D, E, F y AD. (7 términos)
```

```
formula_completa = 'ATT ~ A + B + C + D + E + F + AD'
```

```
model_completo = ols(formula_completa, data=df).fit()
```

```
r2_completo = model_completo.rsquared
```

```

r2_adj_completo = model_completo.rsquared_adj

# --- MODELO COLAPSADO (E eliminado) ---
# Se elimina el factor E.
formula_colapsada = 'ATT ~ A + B + C + D + F + AD'
model_colapsado = ols(formula_colapsada, data=df).fit()
r2_colapsado = model_colapsado.rsquared
r2_adj_colapsado = model_colapsado.rsquared_adj

print("--- Comparación de Modelos para ATT (Colapsando E) ---")

## --- Comparación de Modelos para ATT (Colapsando E) ---
print(f"MODELO COMPLETO (Con E): R-cuadrada={r2_completo:.4f}, R-cuadrada_adj={r2_adj_completo:.4f}")

## MODELO COMPLETO (Con E): R-cuadrada=0.6286, R-cuadrada_adj=0.3036
print(f"MODELO COLAPSADO (Sin E): R-cuadrada={r2_colapsado:.4f}, R-cuadrada_adj={r2_adj_colapsado:.4f}")

## MODELO COLAPSADO (Sin E): R-cuadrada=0.6285, R-cuadrada_adj=0.3808
print("\n--- Conclusión ---")

##
## --- Conclusión ---
r2_change = "aumento" if r2_colapsado >= r2_completo else "bajo"
r2_adj_change = "aumento" if r2_adj_colapsado >= r2_adj_completo else "bajo"

print(f"Cambio en R-cuadrada: {r2_change}")

## Cambio en R-cuadrada: bajo
print(f"Cambio en R-cuadrada (ajustada por g.l.): {r2_adj_change}")

## Cambio en R-cuadrada (ajustada por g.l.): aumento
if r2_change == "bajo" and r2_adj_change == "aumento":
    print("\n El resultado se alinea con la opción: R-cuadrada bajo y R-cuadrada (ajustada por g.l.) aumento."

##
## El resultado se alinea con la opción: R-cuadrada bajo y R-cuadrada (ajustada por g.l.) aumento.

import pandas as pd
import numpy as np
from itertools import combinations

# Datos de la matriz de diseño y la variable de respuesta ATT
data = {
    'A': [-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, 1],
    'B': [-1, -1, 1, 1, -1, -1, 1, 1, -1, -1, 1, 1, -1, -1, 1],
    'C': [-1, -1, -1, -1, 1, 1, 1, 1, -1, -1, -1, -1, 1, 1, 1],
    'D': [-1, -1, -1, -1, -1, -1, -1, -1, 1, 1, 1, 1, 1, 1, 1],
    # E se elimina/colapsa
    'F': [-1, -1, 1, 1, 1, 1, -1, -1, 1, 1, -1, -1, -1, -1, 1],
    'ATT': [6.2, 5.6, 5.8, 5.8, 5.7, 6.4, 6.4, 6.6, 5.3, 6.6, 5.2, 5.5, 6.9, 7.1, 6.7, 6.9]
}

df = pd.DataFrame(data)

```

```

N = len(df)
n_half = N / 2

# Factores en el modelo colapsado (A, B, C, D, F)
factors_retained = ['A', 'B', 'C', 'D', 'F']
interaction_effects_retained = {}

# Calcular las interacciones de 2 factores con los factores retenidos
for f1, f2 in combinations(factors_retained, 2):
    interaction_name = f1 + f2

    # Calcular el vector de signos del efecto
    df[interaction_name] = df[f1] * df[f2]

    # Calcular el efecto: (Suma del producto del vector de interacción por ATT) / (N/2)
    sum_product = (df[interaction_name] * df['ATT']).sum()
    effect = sum_product / n_half

    # Almacenar el efecto
    interaction_effects_retained[interaction_name] = effect

# Crear un DataFrame con los efectos de interacción restantes
effects_df = pd.DataFrame(interaction_effects_retained.items(), columns=['Interacción', 'Efecto'])
effects_df['|Efecto|'] = np.abs(effects_df['Efecto'])
effects_df = effects_df.sort_values(by='|Efecto|', ascending=False).reset_index(drop=True)

# --- Interacciones de 2 Factores Colapsando E (Factores A, B, C, D, F) ---
print(effects_df.head(5))

##   Interacción  Efecto  |Efecto|
## 0          BF  0.4125    0.4125
## 1          CD  0.4125    0.4125
## 2          BD -0.2875    0.2875
## 3          CF -0.2875    0.2875
## 4          AF  0.2625    0.2625

# Identificar el más significativo
most_significant = effects_df.iloc[0]

# Identificar el menos significativo
less_significant = effects_df.iloc[-1]

print(f"El par de efectos (interacción) más significativo es: **{most_significant['Interacción']}** (Ef

## El par de efectos (interacción) más significativo es: **BF** (Efecto = 0.412)
print(f"El par de efectos (interacción) menos significativo es: **{less_significant['Interacción']}** (Ef

## El par de efectos (interacción) menos significativo es: **AC** (Efecto = 0.037)

import pandas as pd
import numpy as np
import statsmodels.api as sm
from statsmodels.formula.api import ols
from itertools import combinations

```

```

# 2. Crear las interacciones principales para el modelo (A, B, C, D, F)
# Incluiremos las interacciones más grandes para el ANOVA: AD, BF, BD, DF, CF
df['AD'] = df['A'] * df['D']
df['BF'] = df['B'] * df['F']
df['BD'] = df['B'] * df['D']
df['DF'] = df['D'] * df['F']
df['CF'] = df['C'] * df['F']

# 3. Formular el Modelo OLS Colapsado (sin E)
# Incluimos los 5 factores principales y las 5 interacciones de mayor magnitud.
formula_anova = 'ATT ~ A + B + C + D + F + AD + BF + BD + DF + CF'
model_anova = ols(formula_anova, data=df).fit()

# 4. Realizar el ANOVA y ordenar por P-valor
anova_table = sm.stats.anova_lm(model_anova, typ=2)

print("--- Tabla ANOVA (Modelo Colapsado sin E para ATT) ---")

## --- Tabla ANOVA (Modelo Colapsado sin E para ATT) ---
# Mostrar solo los factores principales y las interacciones relevantes
relevant_terms = ['A', 'B', 'C', 'D', 'F', 'AD', 'BF', 'BD', 'DF', 'CF']
anova_results = anova_table.loc[anova_table.index.intersection(relevant_terms)]
print(anova_results[['sum_sq', 'F', 'PR(>F)']].sort_values(by='PR(>F)', ascending=False))

##          sum_sq          F    PR(>F)
## F      0.005625    0.039074  0.849829
## B      0.050625    0.351664  0.574828
## D      0.180625    1.254703  0.305480
## AD      0.180625    1.254703  0.305480
## DF      0.225625    1.567294  0.257192
## A      0.330625    2.296671  0.180432
## CF      0.330625    2.296671  0.180432
## BD      0.330625    2.296671  0.180432
## BF      0.680625    4.727931  0.072622
## C      2.805625   19.489146  0.004497

print("-" * 50)

## -----
# 5. Identificar el Factor Principal No Significativo (P-valor más alto)
# Filtramos solo por factores principales
principal_factors_anova = anova_results.loc[['A', 'B', 'C', 'D', 'F']]
factor_no_sig = principal_factors_anova.sort_values(by='PR(>F)', ascending=False).iloc[0].name
p_value_no_sig = principal_factors_anova.loc[factor_no_sig, 'PR(>F)']

# 6. Identificar la Interacción Más Significativa (P-valor más bajo)
# Filtramos solo por interacciones
interaction_terms_anova = anova_results.loc[['AD', 'BF', 'BD', 'DF', 'CF']]
interaccion_mas_sig = interaction_terms_anova.sort_values(by='PR(>F)', ascending=True).iloc[0].name
p_value_mas_sig = interaction_terms_anova.loc[interaccion_mas_sig, 'PR(>F)']

print(f"Factor Principal MENOS Significativo: **{factor_no_sig}** (P-valor = {p_value_no_sig:.4f})")

```



```

## Factor Principal MENOS Significativo: **F** (P-valor = 0.8498)
print(f"Interacción MÁS Significativa: **{interaccion_mas_sig}** (P-valor = {p_value_mas_sig:.4f})")

## Interacción MÁS Significativa: **BF** (P-valor = 0.0726)

if factor_no_sig == 'A' and interaccion_mas_sig == 'AD':
    print("\n La respuesta es la casilla que cruza A:A con AD.")

```

Variable PH

```

import pandas as pd
import numpy as np

# Datos de la matriz de diseño y la variable de respuesta pH
# Usaremos +1 para el nivel Alto (+) y -1 para el nivel Bajo (-)

data = {
    'A': [-1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1, -1, 1],
    'B': [-1, -1, 1, 1, -1, -1, 1, 1, -1, -1, 1, 1, -1, -1, 1, 1],
    'C': [-1, -1, -1, -1, 1, 1, 1, 1, -1, -1, -1, -1, 1, 1, 1, 1],
    'D': [-1, -1, -1, -1, -1, -1, -1, -1, 1, 1, 1, 1, 1, 1, 1, 1],
    'E': [-1, 1, 1, -1, 1, -1, -1, 1, -1, 1, 1, -1, 1, -1, -1, 1],
    'F': [-1, -1, 1, 1, 1, 1, -1, -1, 1, 1, -1, -1, -1, -1, 1, 1],
    'pH': [4.86, 4.86, 4.85, 4.99, 4.94, 4.74, 4.83, 4.85, 4.81, 4.81, 4.98, 4.98, 4.84, 4.85, 4.96, 4.8]
}

df = pd.DataFrame(data)

# Número de corridas
N = len(df)
n_half = N / 2

# Lista para almacenar los efectos
effects = {}

# Calcular el efecto principal para cada factor (A, B, C, D, E, F)
for factor in ['A', 'B', 'C', 'D', 'E', 'F']:
    # Multiplicar el vector del factor por el vector de respuesta (pH)
    sum_product = (df[factor] * df['pH']).sum()

    # Calcular el efecto: (Suma pH(+) - Suma pH(-)) / (N/2)
    effect = sum_product / n_half
    effects[factor] = effect

# Crear un DataFrame de efectos para visualización y ordenamiento
effects_df = pd.DataFrame(effects.items(), columns=['Factor', 'Efecto'])
effects_df['|Efecto|'] = np.abs(effects_df['Efecto'])
effects_df = effects_df.sort_values(by='|Efecto|', ascending=False).reset_index(drop=True)

# --- Efectos Principales Calculados para pH ---
print(effects_df)

##   Factor   Efecto  |Efecto|

```

```
## 0      B  0.07125  0.07125
## 1      C -0.03625  0.03625
## 2      A -0.01875  0.01875
## 3      D  0.01875  0.01875
## 4      F -0.01375  0.01375
## 5      E -0.00625  0.00625
```

Efectos de interacción

```
import pandas as pd
import numpy as np
from itertools import combinations

# Lista para almacenar los efectos de interacción
interaction_effects = {}

factors = ['A', 'B', 'C', 'D', 'E', 'F']
for f1, f2 in combinations(factors, 2):
    interaction_name = f1 + f2

    # Calcular el vector de signos de la interacción
    df[interaction_name] = df[f1] * df[f2]

    # Calcular el efecto
    sum_product = (df[interaction_name] * df['pH']).sum()
    effect = sum_product / n_half

    # Almacenar el efecto
    interaction_effects[interaction_name] = effect

# Crear un DataFrame con todos los efectos de interacción
effects_df = pd.DataFrame(interaction_effects.items(), columns=['Interacción', 'Efecto'])
effects_df['|Efecto|'] = np.abs(effects_df['Efecto'])
effects_df = effects_df.sort_values(by='|Efecto|', ascending=False).reset_index(drop=True)

print("--- Interacciones de 2 Factores (Estimados Confundidos) para pH ---")

## --- Interacciones de 2 Factores (Estimados Confundidos) para pH ---
print(effects_df)
```

```
##      Interacción  Efecto  |Efecto|
## 0             AC -0.05375  0.05375
## 1             BE -0.05375  0.05375
## 2             BC -0.04375  0.04375
## 3             DF -0.04375  0.04375
## 4             AE -0.04375  0.04375
## 5             BD  0.04125  0.04125
## 6             CF  0.04125  0.04125
## 7             AB  0.02875  0.02875
## 8             CE  0.02875  0.02875
## 9             DE -0.02625  0.02625
## 10            AF -0.02625  0.02625
## 11            CD  0.01375  0.01375
## 12            BF  0.01375  0.01375
```

## 13	AD	-0.00875	0.00875
## 14	EF	-0.00875	0.00875